SCR2043 OPERATING SYSTEMS

Name	: GUI KAH SIN	Marks
Student ID	: A23CS0080	
Section	: 02	

This lab assessment is designed to test your understanding and skills on some basic concepts and tools related to process monitoring and management in operating system. Please follow the instructions carefully and submit your answers in this word document and rename the file as **os-lab-assessment02-studentname-matricno.docx**.

Essential Steps Before Starting Lab Assessment 2:

1. **Download necessary source codes:**

   Use the `wget` command to retrieve the following source code files to your Linux (or WSL or MacOS) environment:

```
wget -O mainprocess.c https://rebrand.ly/mainprocess_c
wget -O subprocess1.c https://rebrand.ly/subprocess1_c
wget -O subprocess2.c https://rebrand.ly/subprocess2_c
```

2. **Compile the source files:**

   Use the `gcc` compiler to create executable files from the source code.

```
gcc mainprocess.c -o mainprocess
gcc subprocess1.c -o subprocess1
gcc subprocess2.c -o subprocess2
```

3. **Execute the dummy processes:**

   Run all the dummy processes

```
./mainprocess &
```

   Press **enter** two times.

4. The dummy processes are running for 2 hours. If you took longer than 2 hours on questions 1-9, please restart the main process with `./mainprocess &`.

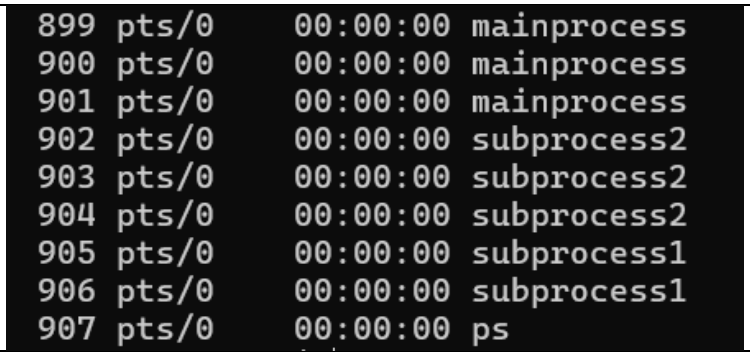
Lab Assessment 2 : Linux Process Monitoring and Management

Instructions:

1. Carefully execute each command as instructed in the questions.
2. Write down the exact command used for each task.
3. Capture a screenshot of the command's output.

Question 1

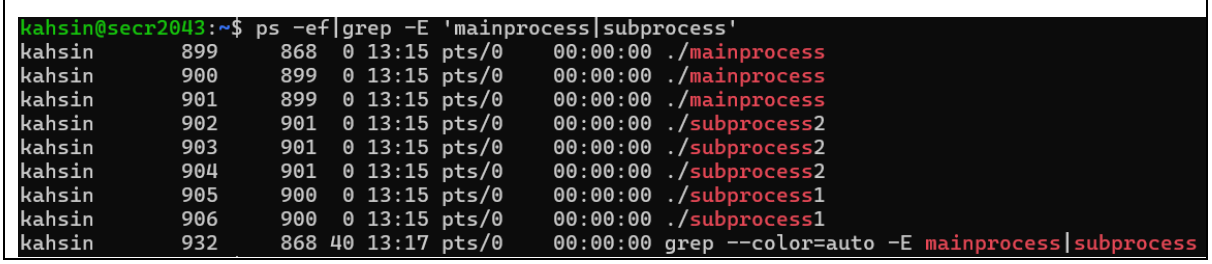
Use the `ps` command with the appropriate option to display a complete list of all running processes within the Linux operating system.

Command
ps -e
Output


```
899 pts/0    00:00:00 mainprocess
900 pts/0    00:00:00 mainprocess
901 pts/0    00:00:00 mainprocess
902 pts/0    00:00:00 subprocess2
903 pts/0    00:00:00 subprocess2
904 pts/0    00:00:00 subprocess2
905 pts/0    00:00:00 subprocess1
906 pts/0    00:00:00 subprocess1
907 pts/0    00:00:00 ps
```

Question 2

Employ the `ps` command with necessary options to unveil comprehensive details about each running process.

Command
ps -ef |grep -E ‘mainprocess|subprocess’
Output


```
kahsin@secr2043:~$ ps -ef|grep -E 'mainprocess|subprocess'
kahsin       899     868  0 13:15 pts/0    00:00:00 ./mainprocess
kahsin       900     899  0 13:15 pts/0    00:00:00 ./mainprocess
kahsin       901     899  0 13:15 pts/0    00:00:00 ./mainprocess
kahsin       902     901  0 13:15 pts/0    00:00:00 ./subprocess2
kahsin       903     901  0 13:15 pts/0    00:00:00 ./subprocess2
kahsin       904     901  0 13:15 pts/0    00:00:00 ./subprocess2
kahsin       905     900  0 13:15 pts/0    00:00:00 ./subprocess1
kahsin       906     900  0 13:15 pts/0    00:00:00 ./subprocess1
kahsin       932     868 40 13:17 pts/0    00:00:00 grep --color=auto -E mainprocess|subprocess
```

Question 3

Use the `ps` command with some tools to only list processes named "subprocess" and show some info about them.

Command
ps -ef| grep -E ‘subprocess’
Output

```
kahsin@secr2043:~$ ps -ef|grep -E 'subprocess'
kahsin       902     901  0 13:15 pts/0    00:00:00 ./subprocess2
kahsin       903     901  0 13:15 pts/0    00:00:00 ./subprocess2
kahsin       904     901  0 13:15 pts/0    00:00:00 ./subprocess2
kahsin       905     900  0 13:15 pts/0    00:00:00 ./subprocess1
kahsin       906     900  0 13:15 pts/0    00:00:00 ./subprocess1
kahsin       934     868 33 13:19 pts/0    00:00:00 grep --color=auto -E subprocess
```

Question 4

Execute the `ps` command, specifying options that reveal only the following columns:

- Process ID (`pid`)
- Owner of the process (`user`)
- CPU percentage (`pcpu`)
- Memory percentage (`pmem`)
- Command (`cmd`)

Command
ps -eo pid,user,pcpu,pmem,cmd | grep -E ‘subprocess’
Output

```
kahsin@secr2043:~$ ps -eo pid,user,pcpu,pmem,cmd | grep -E 'subprocess'
    902 kahsin     0.0  0.1 ./subprocess2
    903 kahsin     0.0  0.1 ./subprocess2
    904 kahsin     0.0  0.1 ./subprocess2
    905 kahsin     0.0  0.1 ./subprocess1
    906 kahsin     0.0  0.1 ./subprocess1
    942 kahsin    50.0  0.1 grep --color=auto -E subprocess
```

Question 5

Building on the ps command used in Question 4, can you add an option to sort the listed processes by their memory usage (`pmem`)?

Command
ps -eo pid,user,pcpu,pmem,cmd --sort=pmem | grep -E 'subprocess'
Output

```
kahsin@secr2043:~$ ps -eo pid,user,pcpu,pmem,cmd | grep -E 'subprocess'
    902 kahsin    0.0  0.1 ./subprocess2
    903 kahsin    0.0  0.1 ./subprocess2
    904 kahsin    0.0  0.1 ./subprocess2
    905 kahsin    0.0  0.1 ./subprocess1
    906 kahsin    0.0  0.1 ./subprocess1
    945 kahsin   33.3  0.1 grep --color=auto -E subprocess
```

Question 6

Construct a command using `ps`, suitable options, and any additional tools to visualize the hierarchical structure (tree-like) of the following processes:

- "mainprocess"
- "subprocess1"
- "subprocess2"

Command
ps --forest -C mainprocess -C subprocess1 -C subprocess2
Output

```
kahsin@secr2043:~$ ps --forest -C mainprocess -C subprocess1 -C subprocess2
    PID TTY          TIME CMD
    899 pts/0    00:00:00 mainprocess
    900 pts/0    00:00:00  \_ mainprocess
    905 pts/0    00:00:00  |   \_ subprocess1
    906 pts/0    00:00:00  |   \_ subprocess1
    901 pts/0    00:00:00  \_ mainprocess
    902 pts/0    00:00:00      \_ subprocess2
    903 pts/0    00:00:00      \_ subprocess2
    904 pts/0    00:00:00      \_ subprocess2
```

Question 7

Use `pstree` command with option that show the number of threads to each process.

Command
pstree -c -s 899
Output

```
kahsin@secr2043:~$ pstree -c -s 899
systemd---sshd---sshd---sshd---bash---mainprocess-+-mainprocess-+-subprocess1
                                                  |             `-subprocess1
                                                  `-mainprocess-+-subprocess2
                                                                |-subprocess2
                                                                `-subprocess2
```

Question 8

Use `renice` command to change priority level of one of process “subprocess1”.

Command
ps -o pid,nice,comm -C subprocess1 sudo renice -10 905
Output

```
kahsin@secr2043:~$ ps -o pid,nice,comm -C subprocess1
    PID  NI COMMAND
    905   0 subprocess1
    906   0 subprocess1
kahsin@secr2043:~$ sudo renice -10 905
[sudo] password for kahsin:
905 (process ID) old priority 0, new priority -10
```

Question 9

Terminate all running processes with the name “mainprocess”.

Command
killall -15 mainprocess
Output

```
kahsin@secr2043:~$ killall -15 mainprocess
Main process (ID: 900) received signal: 15. Terminating...
Main process (ID: 899) received signal: 15. Terminating...
Main process (ID: 901) received signal: 15. Terminating...
[1]+  Done                    ./mainprocess
```

Question 10

Write a short C or Python code (choose only one language) demonstrating multiprocessing with `fork()` and `wait()`. Compile and/or run the code. Show the output.

Source Code:

```
nano multiprocessing.c
gcc -o multiprocessing multiprocessing.c
./multiprocessing
```

multiprocessing.c

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

int main() {
    pid_t pid;

    // Create a child process
    pid = fork();

    if (pid < 0) {
        // Fork failed
        perror("Fork failed");
        exit(1);
    } else if (pid == 0) {
        // Child process
        printf("Child Process: My PID is %d and my parent's PID is %d\n", getpid(),
getppid());
        // Simulate some work in child process
        sleep(2);
        printf("Child Process: Finished its work\n");
    } else {
        // Parent process
        printf("Parent Process: My PID is %d and I created a child process with PID %d\n",
getpid(), pid);
        // Parent waits for child process to finish
        wait(NULL);
        printf("Parent Process: Child finished. Parent exiting\n");
    }

    return 0;
}
```

Output:

kahsin@secr2043: ~

GNU nano 7.2 multiprocessing.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

int main() {
    pid_t pid;

    // Create a child process
    pid = fork();

    if (pid < 0) {
        // Fork failed
        perror("Fork failed");
        exit(1);
    } else if (pid == 0) {
        // Child process
        printf("Child Process: My PID is %d and my parent's PID is %d\n", getpid(), getppid());
        // Simulate some work in child process
        sleep(2);
        printf("Child Process: Finished its work\n");
    } else {
        // Parent process
        printf("Parent Process: My PID is %d and I created a child process with PID %d\n", getpid(), pid);
        // Parent waits for child process to finish
        wait(NULL);
        printf("Parent Process: Child finished. Parent exiting\n");
    }

    return 0;
}
```

Output of the example:

```
kahsin@secr2043:~$ nano multiprocessing.c
kahsin@secr2043:~$ gcc -o multiprocessing multiprocessing.c
kahsin@secr2043:~$ ./multiprocessing
Parent Process: My PID is 970 and I created a child process with PID 971
Child Process: My PID is 971 and my parent's PID is 970
Child Process: Finished its work
Parent Process: Child finished. Parent exiting
```